

Interactive Language-Queryable Gaussian Scenes

James Conant

Sergejs Zahovskis

Dmitry Knorre

Yael Fassbind

Supervised by Alexander Veicht

1. Introduction and motivation

Most virtual reality applications rely on controllers to input or manipulate a scene. However, this can be quite cumbersome, especially when dealing with text input. These constraints limit the immersiveness and user-friendliness of such applications.

Natural language interactions present an opportunity for users to directly express their intent and perform more complex scene manipulations more easily. However, to enable this, 3D scenes must have a semantic understanding of the environment. Otherwise, it is not possible to get a meaningful link between objects and natural language.

The goal of this project is to enable users to manipulate and query 3D scenes using natural language commands such as “highlight the chair.” This involves extending 3D scenes with semantic meaning and language-based querying while maintaining reasonable performance.

1.1. Hardware challenge

Building such an application for a VR headset is challenging due to the limited memory and computational resources available. To be specific, our target headset, the Meta Quest 3, has 8 GB of RAM and an Adreno 740 GPU, which is more suitable for mobile phone workloads than desktop processing [2]. Rendering 3D Gaussian scenes with language features is a computationally intensive task that is high on memory due to the explicit representation of the scene. Working around these constraints is the principal challenge of our project.

2. Prior Work

2.1. 3D Gaussian Splatting (3DGS)

3D Gaussian Splatting (3DGS) was introduced by Kerbl et al. [8] and enables high-quality, real-time rendering of complex scenes. 3DGS are made up of millions of colored, anisotropic 3D Gaussian “blobs” which float in space, creating a 3D scene. Each Gaussian is represented using its position, covariance (defines the shape, scale and orientation), color and opacity.

To render the 3DGS, the 3D Gaussians are projected into 2D as elliptical splats. From the viewpoint out, the splats are alpha-blended in depth order, resulting in the 2D view.

Using 3DGS, one can reconstruct a very high-quality and realistic 3D scene only using a few images.

2.2. Occam’s Language Gaussian Splatting

Occam’s LGS: An Efficient Approach for Language Gaussian Splatting [4] introduces a simple method which adds language features to 3D Gaussians while removing the complex modules or training seen in other approaches such as Langsplat [13].

In Occam, a given 3DGS scene is provided with semantic understanding by adding a high-dimensional language feature to each Gaussian in the scene.

Conceptually, Occam works as follows. First, the 3DGS is forward rendered from several different viewpoints. For each of those 2D images, the language features are extracted for all pixels.

To calculate the language embedding for a single Gaussian, the set of all views where said Gaussian is visible are identified. For all those views, the pixel language features are mapped to the corresponding Gaussian. Finally, the language feature of the Gaussian is calculated using the weighted average of all pixel language features using the alpha blending weight from each viewpoint.

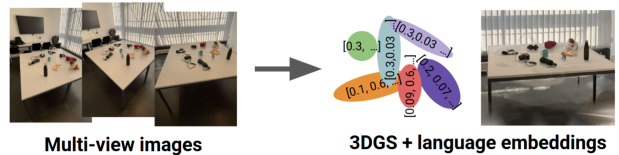


Figure 1. 3DGS with language features

3. Methodology

3.1. Rendering

Our main challenge to overcome was finding a way to render the language feature field in real time with a low

memory footprint. As the papers suggest, we started by trying to render the full alpha-blended language feature vector per pixel. However, this attempt ended up being infeasible because of the limitations of graphics shader architectures. Shaders are optimized for efficiently rendering vectors of at most 4 values, as in the RGBA format, but the language features we were dealing with held at least 512 values.

While feature compression could be done with lightweight techniques such as PCA or more involved approaches like LangSplatV2, the approach would require multiple shader passes per frame. Furthermore, in the context of a VR headset application, rendering must be done separately for each eye, which effectively doubles the rendering workload.

We went back to the drawing board and reexamined the feature alpha blending equation introduced by the original LangSplat paper [13].

$$F(v) = \sum_{i \in \mathcal{N}} \mathbf{f}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

where $F(v)$ is the language feature rendered for pixel v , $\mathbf{f}_i$ is the language feature for the Gaussian splat i , and $\alpha = o_i G_i^{2D}(v)$, which models the alpha blending contribution of the Gaussian splat via its opacity and the 3D Gaussian projected rasterized to pixel space. The key insight was as follows. For our application, the only thing we planned to do with the language feature field in the end was perform cosine similarity over the whole frame to highlight objects matching the user’s natural language query transformed to a CLIP vector $\mathbf{q}$, getting a single scalar $\mathbf{q} \cdot F(v)$ for each pixel. Instead of needing to render the full language feature field, what if instead we precomputed the cosine similarity of the query with each Gaussian splat language feature in the scene, and then only rendered the cosine similarity output scalars for each Gaussian?

$$\mathbf{q} \cdot F(v) = \mathbf{q} \cdot \left(\sum_{i \in \mathcal{N}} \mathbf{f}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \right) \quad (1)$$

$$= \sum_{i \in \mathcal{N}} \mathbf{q} \cdot \left(\mathbf{f}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \right) \quad (2)$$

$$= \sum_{i \in \mathcal{N}} (\mathbf{q} \cdot \mathbf{f}_i) \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (3)$$

To implement this, we could precompute a buffer of the cosine similarities $\mathbf{q} \cdot \mathbf{f}_i$ between the current user query and each Gaussian in the scene, then during real time rendering, we would only have to alpha-blend a single scalar value from each Gaussian. There would be some latency after each user query to compute this buffer, but we would only

need one shader pass per eye for each frame, thus making this no slower than standard RGBA rendering, which we verified as working in real time on the headset for small scenes. Our implementation sends the user query from the headset to the server, performs the similarity scores with each Gaussian, then sends the buffer back so the headset can render the similarities in real time.

To cut down on the noisiness of our results, our supervisor suggested we use relevancy scores as proposed in the literature [9] [13]. The idea is to also perform cosine similarities with a small set of canonical queries such as *object*, *material*, *thing*, etc. Then, take the softmax between the user query and each canonical query individually, taking the minimum as the final relevancy score. Scores can be discarded if they are less than 0.5. To illustrate why this works, imagine the user queried *airplane*. The relevancy score essentially answers the question: is this item in the scene more of an *airplane* than it is a generic object? If we made no modifications to the language feature alpha blending equation, computing this would be more involved because we would have to render the user query similarity and the 3-4 canonical similarities for each pixel.

This challenge was addressed by following our supervisor’s suggestions of precomputing the relevancy scores directly for each Gaussian in the scene and then alpha blending those together.

This yielded good results in practice, and thus it was used in the final version of the system. Following is the equation of the relevancy score as in LERF, where ϕ_{qry} is the CLIP embedding of the user query, ϕ_{img} is the rendered language embedding, and ϕ_{canon}^i is a CLIP embedding of one of the canonical queries.

$$s = \min_i \frac{\exp(\phi_{img} \cdot \phi_{qry})}{\exp(\phi_{img} \cdot \phi_{qry}) + \exp(\phi_{img} \cdot \phi_{canon}^i)}$$

3.2. Architecture

The key factor that shaped our decision-making for our application architecture was the limited memory and compute available on the headset. We tried to offload as much computation as possible from the headset to a laptop connected over a cable or local network. Initially, we investigated offloading the rendering itself to the laptop, which is common in VR games with heavy rendering loads. It became quickly evident that the available hardware did not meet the OS and performance requirements for this task. Most of the server-side apps capable of interfacing with the headset require a Windows laptop with a GPU, which was not available to any member of the team. Consequently, we proceeded under the hypothesis that we could perform the rendering directly on the headset, while offloading all remaining heavyweight computation to a server. This approach was supported by early experiments of rendering

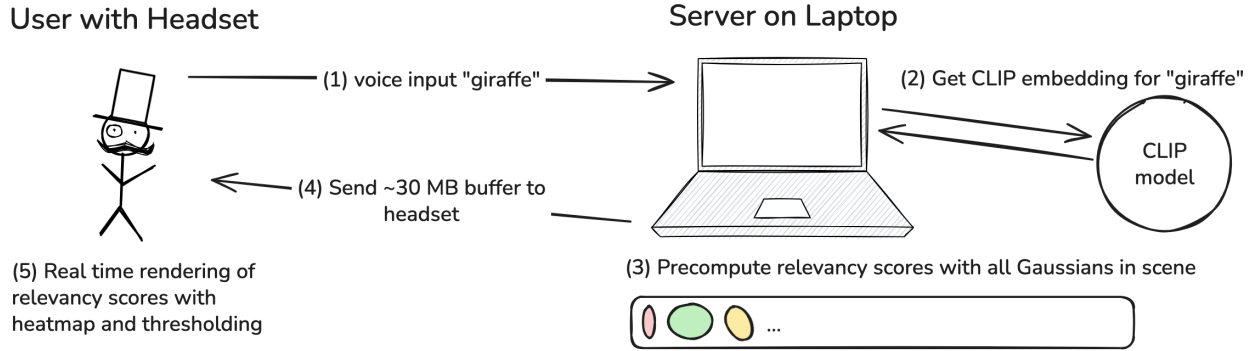


Figure 2. Simplified Architecture Diagram

Gaussian Splat scenes on the headset. Fortunately, this worked fairly well without needing much optimization.

In our final architecture, we deploy an application to the headset that handles the user input and performs rendering. It communicates with a server running on a nearby laptop. When the user issues a query, the voice input is sent to the server. The server transcribes the voice input into natural language and runs a CLIP model over the input to extract a 512-dimensional embedding. The next computation is precomputing the relevancy scores for every Gaussian Splat in the scene. To make this possible, both the server and the headset maintain a copy of the same Gaussian scene in memory. The server also precomputes the buffers of cosine similarities with the canonical embeddings *object*, *thing*, etc. We create a buffer of size $N * 4$ bytes, where N is the number of Gaussians and 4 is the size of the minimal float representation we use. Then, we compute all the relevancy scores, store them in the buffer, and send them back to the headset application. The rendering runs on the GPU as a series of shaders. We copy over the buffer from the CPU to GPU memory, and after a few frames, our custom shaders pick up on the new values and show the highlighted object of the user’s interest.

3.3. Creating own 3D Gaussian scenes with language embeddings

To create a 3D Gaussian scenes, we used the guide [18] by Jonathan Stephens linked on the official implementation of *3D Gaussian Splatting for Real-Time Radiance Field Rendering* [7]. First, one starts with capturing the scene which should be converted to a 3DGS scene. This can be done by either recording a video of the scene or taking several photos with lots of overlap between different views. Before those images can be used to train the scene, they have to be prepared because the optimizer expects a

COLMAP dataset. In the COLMAP dataset, we additionally have the estimated camera poses for each input image as well as a sparse point cloud which is used to initialize the Gaussians. In the second step, this dataset is used to train the scene in order to create the the 3D Gaussian scene.

Finally, this scene has to be extended with language features. Initially, we planned to do this with LangSplatV2, but limited cluster resources prevented us from using that approach. Following our supervisors’ suggestion, we looked into Occam’s Language Gaussian Splatting (LGS) approach. Using his implementation of the approach based on the Occam paper, we were finally able to extend the 3D Gaussian scene with language features. The implementation uses the gsplat library [19] to generate the Gaussian splat representation of a 3D scene. Separately, the language features for each available view are precomputed using the CLIP neural network. Finally, language features are lifted up to compute the language feature for each Gaussian splat, as described in the Occam paper [4]. We further improved the first stage by introducing an option to prune the Gaussian splats using MCMC pruning strategy [10], which allowed us to control the number of Gaussian splats in the final 3D scene.

4. Implementation

Our starting point was a Toy Gaussian Splatting Github repository [12] that claimed to work on the Meta Quest. The repo contains tools for importing Gaussian Splats into Unity Assets and an implementation of the Gaussian Splat rendering algorithm of principal interest to us. We modified this implementation and deployed it to the headset, where it ran with minimal changes. Reimplementing the renderer seemed unnecessary, therefore we forked the repository and used it as a foundation upon which our application was

built.

Here is our team’s [Github repository](#).

To implement our custom rendering, we first created a GPU buffer that contains N relevancy scores for each Gaussian. The server will provide this to the headset after each query. We modified the existing Gaussian splatting RGB shader to blend relevancy scores instead. It outputs the blended relevancy score in the red channel of the output texture and the grayscale intensity in the green channel. Then, we wrote a custom compute shader to pass over all the pixels in the output texture and convert the relevancy scores to a heatmap color if the score is greater than some threshold and leave it as grayscale otherwise. To assist with the heatmap coloring, we also have the server return the minimum and maximum relevancy score to save us a $O(H \cdot W)$ extrema computation on the headset. The threshold we use is 0.5. If the relevancy score is greater than 0.5, we normalize it with the minimum and maximum relevancy scores and then plug it into a simple linear interpolation-based color map for the final color.

This implementation allows us to achieve roughly the same speed as the default RGB rendering. The disadvantage is that it does not render color, because we have only 4 channels and 5 values per pixel we want to render: r, g, b, a, RS , where RS is the relevancy score. In our initial demo we render everything else as grayscale. For the final version of our app, we added a mode that does two draw passes to render both color and the highlighted Gaussians.

The server is running on a laptop that the Meta Quest connects to over WiFi. It is a simple Python server with the following functions:

1. Use OpenAI’s Whisper model [15] for speech-to-text upon receiving raw audio input from the Unity app over the network.
2. Read in a `.ply` file of the same Gaussian splat scene running on the headset.
3. Compute CLIP embeddings using the ViT-B-16 architecture, the `open_clip` package, and `laion2b_s34b_b88k` pretrained weights [5] [14] [6] [17].
4. Compute a buffer of N relevancy scores with respect to the canonical embeddings of *object*, *thing*, *texture*, *material* and stream it back to the headset along with the maximum and minimum score for the given query.

For our demo day app, we used the `bonsai` scene from the NERF baselines paper [11]. We fit the Occam language features using a pipeline running on the student cluster, and exported the final scene as a `.ply` file with 512 additional language feature fields. Our supervisor provided us his personal GitHub repository with the implementation of the Occam paper. We split those out because the Unity app cannot

fit that big of a file into memory and only the Python server needs access to those fields.

The number of Gaussians proved to be a key bottleneck for the rendering speed of our app. We used scenes with around a million Gaussians, because more than that slowed the rendering to unusable speeds. When fitting our custom scene, we experimented with methods to reduce the number of Gaussians, including the MCMC pruning strategy mentioned in 3.3. The results looked blurrier but it cut the number of Gaussians in the scene representation significantly.

For the application itself, we implemented a simple UI with two input methods: voice input and keyboard controls. Both are activated by triggers on the Meta Quest controllers. We use built in Unity UI components and the XR package. After getting feedback on our app, we removed the keyboard controls so only voice input is supported. Although keyboard input is more accurate, we found that the voice input is still reliable and accurate enough.

To prevent visual artifacts when the camera is close to or inside Gaussian splats, we implemented near-plane culling in the vertex shaders. Splats closer than a configurable threshold distance are discarded, eliminating flickering that would otherwise occur at close range. This is particularly important for VR applications where users may move freely through the scene.

5. Results

The final system implements a fully functional system that runs at a stable frame rate on Meta Quest and allows the user to query the scene using voice input. Based on the user query, the relevancy scores for each Gaussian are calculated and highlighted accordingly.

The system provides three rendering modes: (1) a standard mode that displays colored Gaussian Splats, (2) Occam Similarities mode, which displays the similarity score between the query and the language embeddings as a grayscale scene with heatmap highlighting, and (3) Occam Colored Similarities mode, which overlays the relevancy heatmap on top of the original colored Gaussian Splats. The third mode uses a multi-pass rendering approach: first rendering the colored splats to a temporary render target, then rendering the occam similarities to a separate target, and finally compositing both using alpha blending. This allows users to see the original scene colors while still identifying highlighted objects.

The rendering mode can be changed using the Meta Quest controllers. The right trigger toggles between Gaussian Splats and Occam Similarities modes, while the A button on the right controller activates the Occam Colored Similarities mode. The current rendering mode is displayed in a panel at the top-left corner of the user’s view.

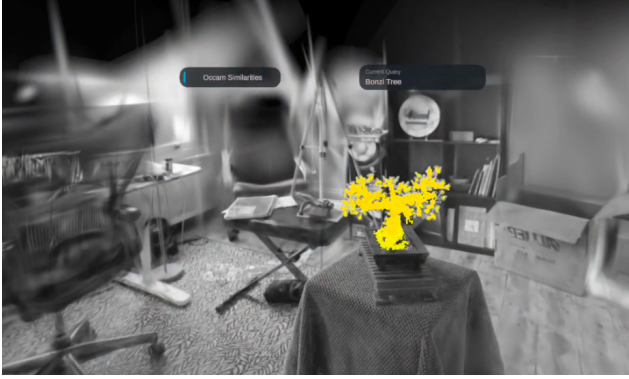


Figure 3. Query "Bonsai Tree" in Occam similarities rendering mode

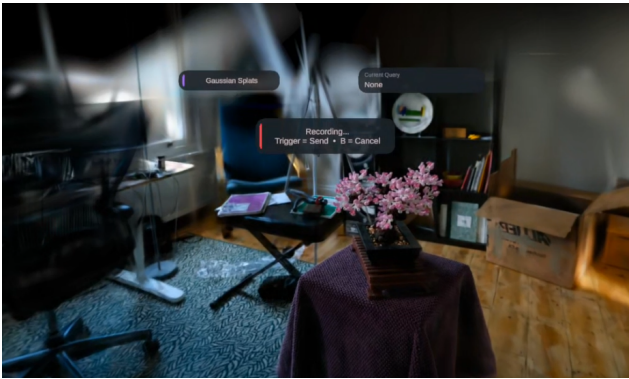


Figure 4. Rendering Gaussian Splats

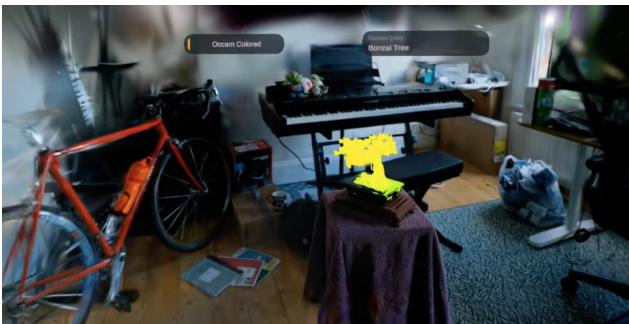


Figure 5. Query "Bonsai Tree" in colored Occam similarities rendering mode



Figure 6. Voice Input

In addition to the application, a custom 3D Gaussian scene with language embeddings was created. It displays a meeting room with a table full of objects which can be queried.

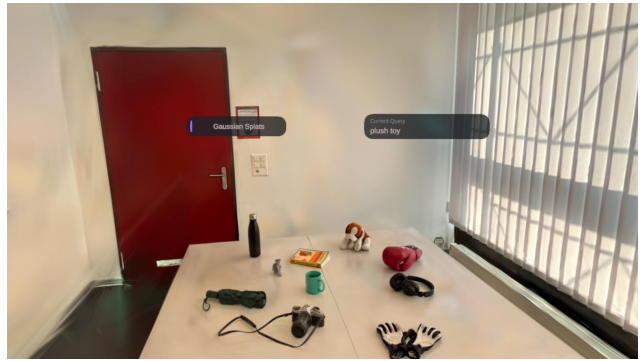


Figure 7. Custom scene

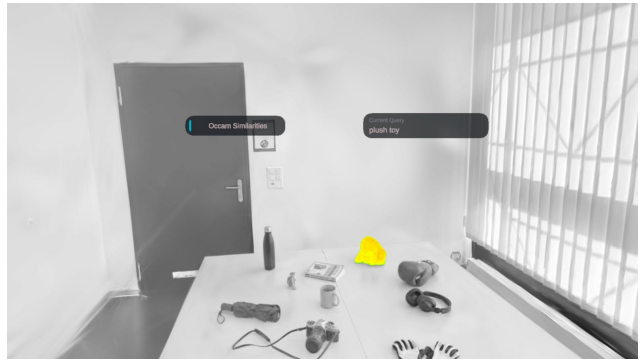


Figure 8. Query "push toy"

6. Evaluation

In order to evaluate the quality of our application, a user study was conducted, which involved 12 participants.

The questionnaire was designed to answer three research questions: (1) How usable is the app? (2) How pleasant is the app? (3) What input type is preferred, voice or text?

To ensure that the study was concise, we omitted the SSQ (Simulator Sickness Questionnaire) and chose a few key questions to determine the users' input preference. The final questionnaire consists of:

- Input preference (voice or text)
- System Usability Scale (SUS)
- User Experience Questionnaire (UEQ-S)
- Overall experience

Two standardized tests SUS [3] and UEQ-S [16] were used to address the first two research questions. Both were selected for their compactness, which allows the participants to quickly give feedback, without too much time investment.

6.1. SUS

SUS has been used for wide variety of projects in research and industry and gives a single number that represents a composite measure of the overall usability of the system under evaluation.

The SUS score was calculated according to the procedure described in the paper. The score can range from 0 to 100, where 100 indicates the best possible value. Our system achieved a **SUS score of 78**, which represents good overall usability.

6.2. UEQ-S

UEQ-S measures the user experience of a system by looking at 8 qualities, which are split into pragmatic and hedonic qualities.

The *pragmatic qualities* display how the application supports users in achieving their goals: *obstructive* or *supportive*, *complicated* or *easy*, *inefficient* or *efficient* and *confusing* or *clear*.

The *hedonic qualities* display how enjoyable the application feels to the user: *boring* or *exciting*, *not interesting* or *interesting*, *conventional* or *inventive* and *usual* or *leading edge*.

Using the data analysis tool provided on the website [1], three main scores were calculated. The values range from -3 to 3, where -3 indicates extremely poor and 3 indicates extremely good performance.

Both the pragmatic and hedonic scores are positive and indicate a good user experience. In particular, the system stands out as being perceived as very interesting.

Quality Type	Mean Score
Pragmatic	1.542
Hedonic	1.583
Overall UX	1.563

Table 1. UEQ-S Results

Quality	Mean Score
obstructive – supportive	1.8
complicated – easy	1.8
inefficient – efficient	0.7
confusing – clear	1.9
boring – exciting	1.4
not interesting – interesting	2.3
conventional – inventive	1.4
usual – leading edge	1.3

Table 2. UEQ-S Quality Scores

Input Method	Percentage
Voice Input	72.7%
Virtual Keyboard	27.3%

Table 3. Overall Preferred Input Method

6.3. Preferred Input

As can be seen in Table 3 and Figure 9, the preferred input mode is voice input. The participants found that voice input is faster, felt more natural and was physically more comfortable. However, in terms of accuracy, the keyboard still dominates.

6.4. Overall Experience

Most users found the application pleasant to use. Some pointed out that the keyboard can be a bit jittery and the occasional camera flickering is awkward. Additionally, some mentioned that the visual quality of the 3DGS scenes could be improved.

Regarding simulation sickness, only one participant reported that they felt slightly motion sick, while all other participants noted no discomfort.

6.5. Summary

Overall, the evaluation results are positive and allow the three research questions to be answered.

The first research question can be answered based on the SUS score of 78, which indicates good overall usability. The second research question can be addressed using the UEQ-S results. Both pragmatic and hedonic quality scores are positive, which indicates an engaging and enjoyable experience. The last question concerns the preferred input mode. Most users leaned towards voice input, although the

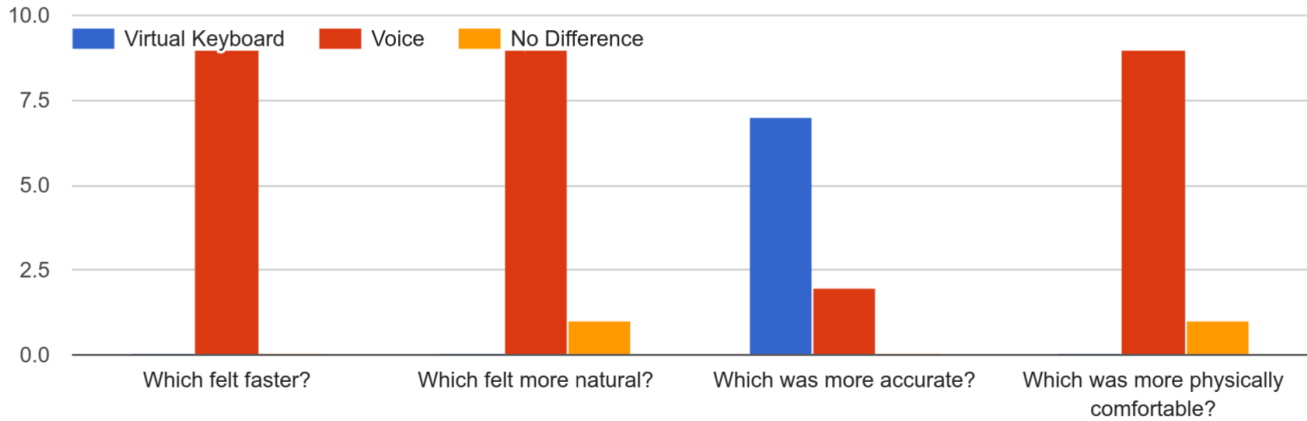


Figure 9. Input comparison

keyboard was perceived as more accurate.

In summary, the evaluation shows that the application is usable, pleasant and voice input is the preferred input type.

It is important to point out that the responses for this questionnaire relate to the application which was presented at the poster presentation. Since then, the quality of the system was improved further:

- Removed keyboard input
- Refined UI panels to improve visibility and usability
- Added Occam Colored Similarities rendering mode that overlays relevancy heatmap on colored Gaussian Splats
- Implemented near-plane culling to prevent flickering when camera is close to objects

Based on these improvements, we would expect the scores for SUS as well as UEQ-S to increase.

7. Limitations and discussion

The main limitation of this project was hardware. As mentioned before, VR headsets like the Meta Quest, have limited memory and computational resources. Because of this, the application depends on an external server for scene querying. This introduces additional latency and a stable connection to the server has to be maintained.

Also, due to the hardware constraints, the performance is still limited. While real-time rendering was achieved, the frame rate is still not ideal. Additionally, the frame rate scales badly with the number of Gaussians in the scene, so it would be great to alleviate this dependency so we can view higher quality scenes.

Another limitation concerns the visual quality of the system. In the standard Occam Similarities mode, scenes are rendered in grayscale while highlighting queried objects. This limitation is addressed by the Occam Colored Similarities mode, which preserves the original scene colors while overlaying the relevancy heatmap. However, this mode incurs a performance cost due to the multi-pass rendering approach (approximately 2x the rendering workload). Additionally, object highlighting can be noisy in some cases.

Despite these limitations, the system provides a solid foundation which can be built upon and create systems which enable applications with natural language based scene interactions. One possible application would be the implementation of a "Find the Difference" game.

Strengths

- Real-time rendering of any 3DGS scene of appropriate size natively on VR headset
- Easy and intuitive natural language interaction using voice input
- Easily extendable pipeline

Weaknesses

- Limited performance and frame rate (12 FPS in standard modes, lower in colored overlay mode)
- Dependent on external server for querying
- Multi-pass rendering in Occam Colored Similarities mode reduces frame rate and works consistently only on smaller scenes

References

- [1] User experience questionnaire (ueq). <https://www.ueq-online.org/>. 6
- [2] Meta quest 3 – full specification - vrcompare. <https://vr-compare.com/headset/metaquest3>, 2026. Accessed: 2026-01-12. 1
- [3] John Brooke. Sus: A quick and dirty usability scale. *Usability Eval. Ind.*, 189, 11 1995. 6
- [4] Jiahuan Cheng, Jan-Nico Zaech, Luc Van Gool, and Danda Pani Paudel. Occam’s lgs: An efficient approach for language gaussian splatting. In *36th British Machine Vision Conference 2025, BMVC 2025, Sheffield, UK, November 24-27, 2025*. BMVA, 2025. 1, 3
- [5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. 4
- [6] Gabriel Ilharco, Mitchell Wortsman, Ross Wightman, Cade Gordon, Nicholas Carlini, Rohan Taori, Achal Dave, Vaishaal Shankar, Hongseok Namkoong, John Miller, Hananeh Hajishirzi, Ali Farhadi, and Ludwig Schmidt. Openclip, July 2021. If you use this software, please cite it as below. 4
- [7] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. Official authors implementation of ”3d gaussian splatting for real-time radiance field rendering”. <https://github.com/graphdeco-inria/gaussian-splatting?tab=readme-ov-file>. 3
- [8] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), July 2023. 1
- [9] Justin* Kerr, Chung Min* Kim, Ken Goldberg, Angjoo Kanazawa, and Matthew Tancik. Lerf: Language embedded radiance fields. In *International Conference on Computer Vision (ICCV)*, 2023. 2
- [10] Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Jeff Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo, 2024. 3
- [11] Jonas Kulhanek and Torsten Sattler. Nerfbaselines: Consistent and reproducible evaluation of novel view synthesis methods, 2025. 4
- [12] Aras Prancėvičius. Gaussian splatting playground in unity. <https://github.com/aras-p/UnityGaussianSplatting>, 2025. 3
- [13] Minghan Qin, Wanhua Li, Jiawei Zhou, Haoqian Wang, and Hanspeter Pfister. Langsplat: 3d language gaussian splatting. *arXiv preprint arXiv:2312.16084*, 2023. 1, 2
- [14] Alec Radford, Jong Wook Kim, Chris Hallacy, A. Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, 2021. 4
- [15] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2022. 4
- [16] Martin Schrepp, Andreas Hinderks, and Jörg Thomaschewski. Design and evaluation of a short version of the user experience questionnaire (ueq-s). *International Journal of Interactive Multimedia and Artificial Intelligence*, 4:103, 01 2017. 6
- [17] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade W Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa R Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. LAION-5b: An open large-scale dataset for training next generation image-text models. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022. 4
- [18] Jonathan Stephens. 3d gaussian splatting for real-time radiance field rendering - detailed windows install usage instructions. <https://github.com/jonstephens85/gaussian-splatting-Windows>. 3
- [19] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025. 3